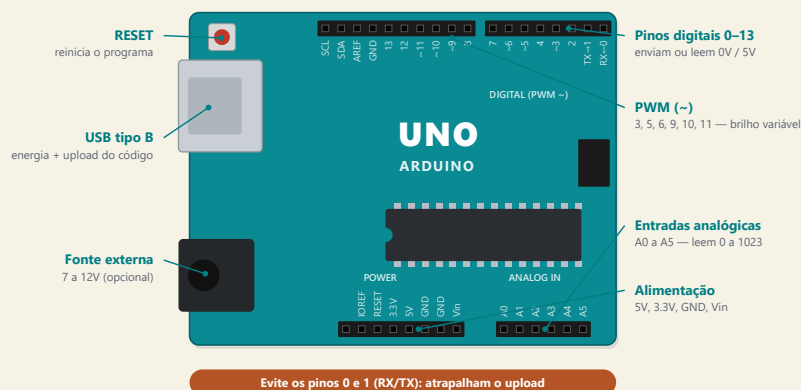
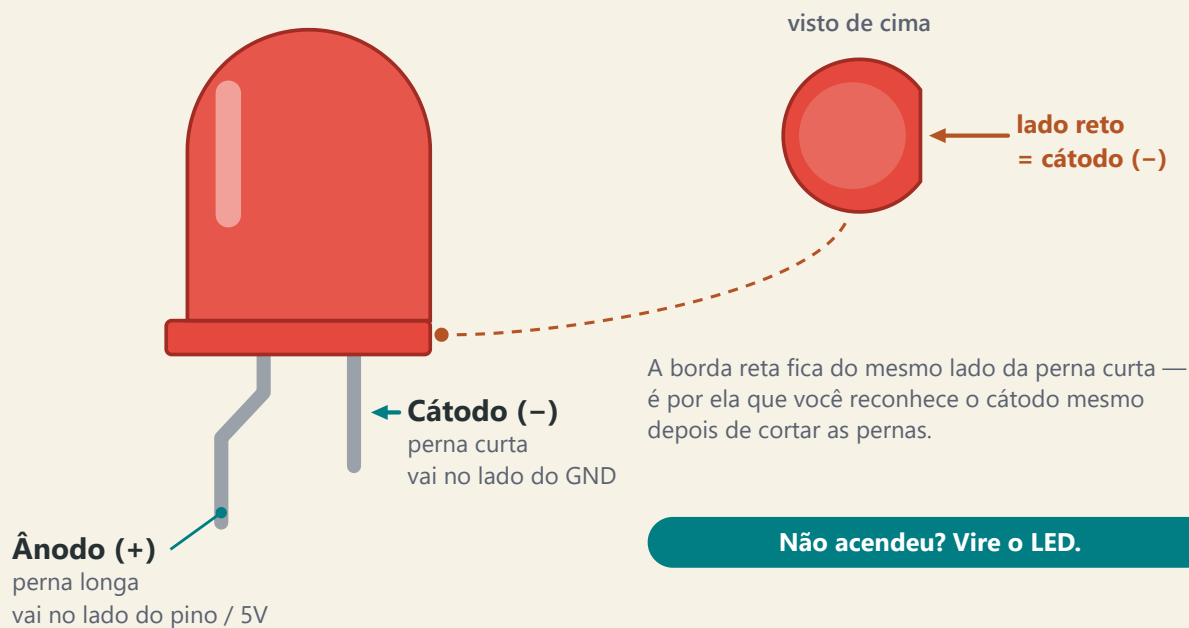


Anexo A — Guia ilustrado dos componentes

Como reconhecer cada peça do kit, entender o que ela faz por dentro e ligá-la sem errar. Guarde este anexo: ele responde 90% das dúvidas de montagem.



A.1 • LED — Light Emitting Diode

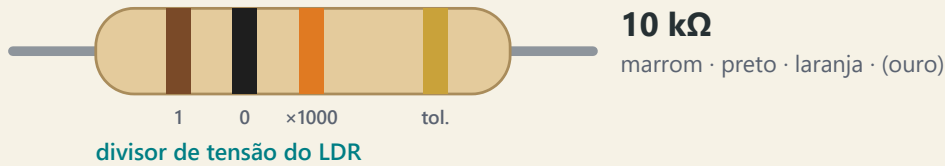
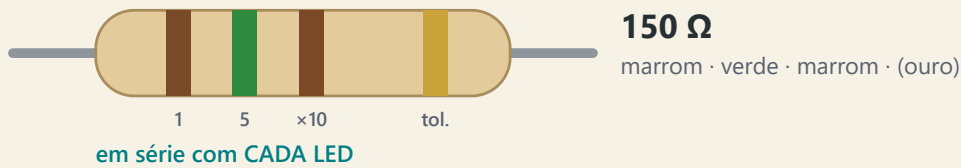


Como identificar	O que significa
Perna mais longa	Ânodo (+) — vai para o pino do Arduino / 5V
Perna mais curta	Cátodo (-) — vai para o GND
Lado do corpo achatado	Confirma o cátodo (use quando as pernas já foram cortadas)
Peça maior lá dentro (a "bigorna")	É o cátodo — dá para ver contra a luz

Como funciona: o LED é um **diódo** — só deixa a corrente passar num sentido. Quando ela passa, a energia dos elétrons é liberada como **fóton**. Por isso ele tem lado certo, e por isso a cor depende do material, não da tinta do corpo.

⚠ Sempre em série com um resistor. O LED não limita a própria corrente: se você deixar, ele puxa tudo que o pino conseguir dar — e queima.

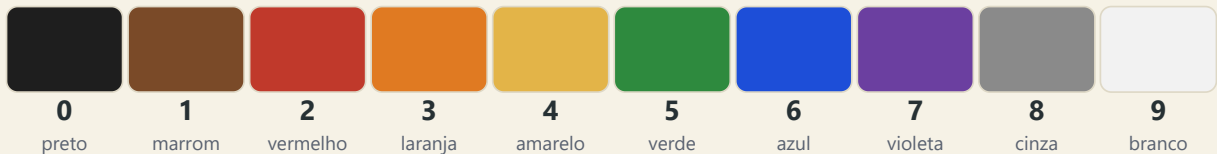
A.2 · Resistor



Como ler: 1ª faixa = 1º dígito · 2ª faixa = 2º dígito · 3ª faixa = quantos zeros
marrom=1 · preto=0 · vermelho=2 · laranja=3 · amarelo=4 · verde=5 · azul=6 · violeta=7

Como ler as faixas: 1ª = primeiro dígito · 2ª = segundo dígito · 3ª = **quantos zeros** · 4ª (dourada/prateada) = tolerância, ignore.

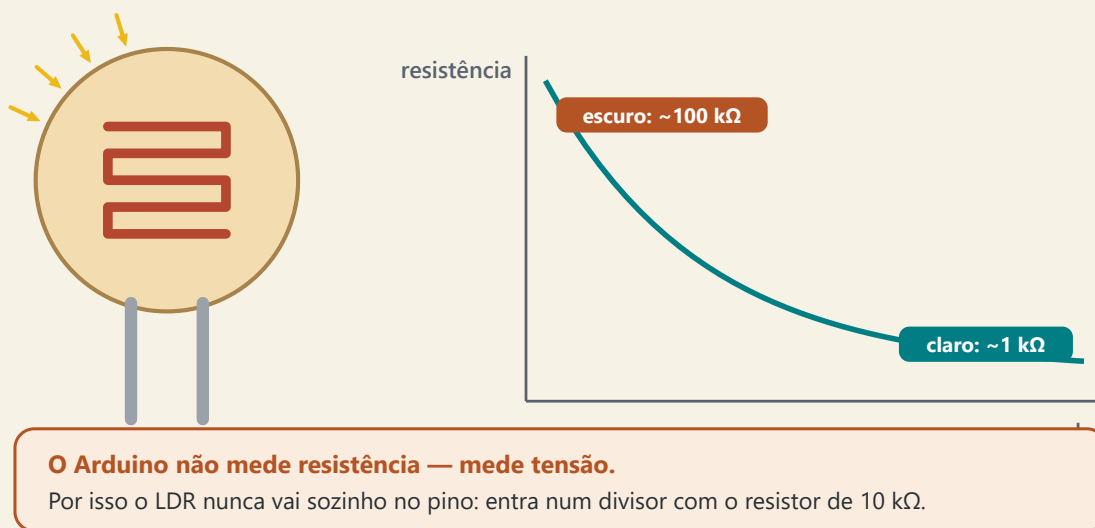
Cada cor é um número



Exemplo: marrom(1) · verde(5) · marrom(1 zero) → 1 5 0 → 150 Ω

Como funciona: o resistor converte parte da energia elétrica em calor, "estreitando o cano". Não tem polaridade — pode ser montado em qualquer sentido.

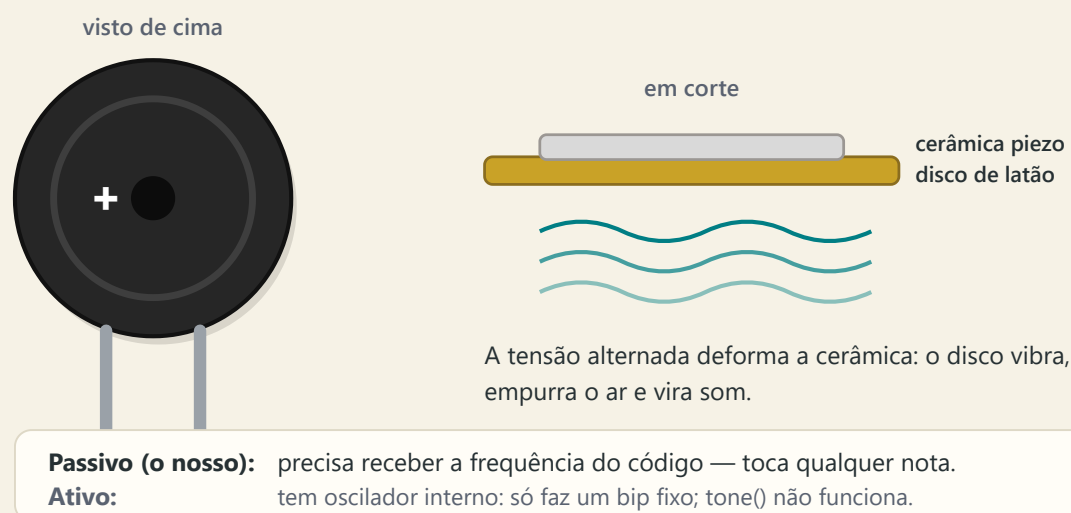
A.3 · LDR — Light Dependent Resistor (fotoresistor)



Como funciona: a superfície serpenteada é de **sulfeto de cádmio**. Quando a luz bate, ela libera elétrons no material e a **resistência despenca**. No escuro, os elétrons ficam presos e a resistência sobe. É um resistor que "sente" luz.

⚠ O Arduino **não mede resistência** — mede tensão. Por isso o LDR nunca vai sozinho no pino: ele entra num **divisor de tensão** com o resistor de 10 kΩ (veja a Prática 4).

A.4 · Buzzer piezoelétrico

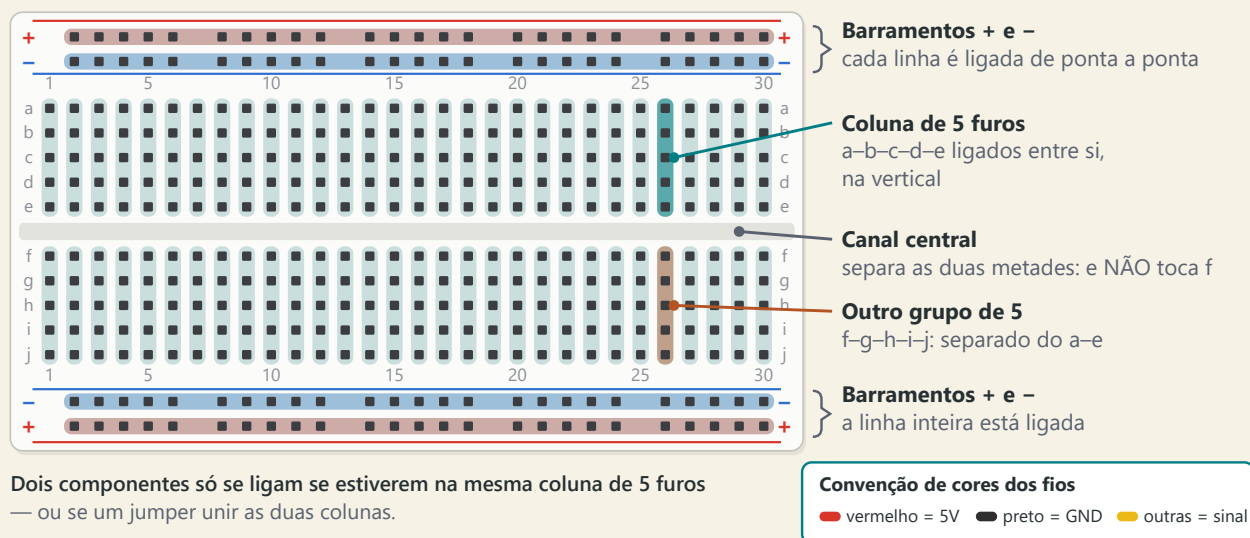


Como funciona: cerâmica piezoelétrica **muda de forma** quando recebe tensão. Alternando a tensão milhares de vezes por segundo, o disco vibra, empurra o ar e gera som. A **frequência** dessa alternância é a **altura da nota** (em Hz).

Passivo (o nosso)	precisa receber a frequência do código — toca qualquer nota
Ativo	tem oscilador interno — só faz um bip fixo, <code>tone()</code> não funciona

Consome pouquíssima corrente: **não precisa de resistor**. Se estiver alto demais na sala, coloque 100 Ω em série para abafar.

A.5 • Protoboard — onde quase todo erro acontece

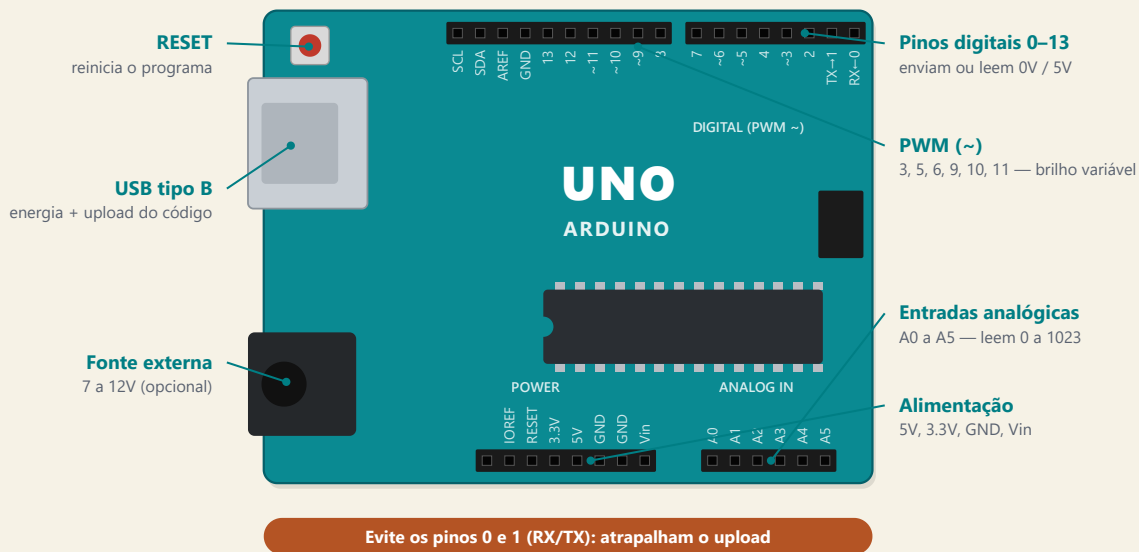


As três regras que resolvem 90% dos erros de montagem:

Regra	Consequência prática
Linhas + e - (horizontais) são ligadas de ponta a ponta	leve o 5V e o GND uma vez só, e todo mundo se serve delas
Colunas centrais ligam 5 furos na vertical	é assim que dois componentes se "encontram"
O canal central separa os dois lados	o furo e não está ligado ao furo f

⚠ Muita protoboard tem o barramento interrompido no meio. Se metade do circuito não liga, passe um jumper unindo as duas metades da linha + e da -.

A.6 · Arduino Uno — o mapa dos pinos



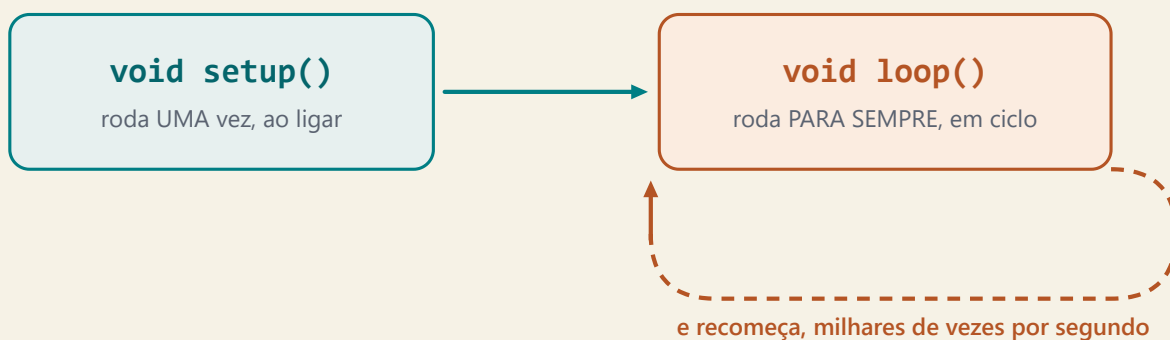
Grupo	Pinos	O que fazem
Digitais	0 a 13	enviam ou leem apenas 0V ou 5V
PWM (com ~)	3, 5, 6, 9, 10, 11	também simulam valores intermediários (0–255)
Analógicos	A0 a A5	leem tensão contínua e devolvem 0 a 1023
Alimentação	5V, 3.3V, GND, Vin	energia para os componentes
Evite	0 e 1 (RX/TX)	são a comunicação com o PC — atrapalham o upload

Limites elétricos que importam: **20 mA** recomendados por pino (40 mA é o máximo absoluto) e **~200 mA** somando todos os pinos. Motor ou fita de LED **nunca** direto no pino: precisa de alimentação externa.

Anexo B — As funções do Arduino, ilustradas

Cada função vem com: o que faz, a sintaxe, um diagrama do que acontece no pino, um exemplo real do curso e o erro que todo mundo comete.

B.1 · Estrutura mínima de todo programa



```
void setup() {  
  // configurações: roda UMA vez  
}  
  
void loop() {  
  // a lógica do robô: roda PARA SEMPRE  
}
```

B.2 · `pinMode()` — dizer se o pino fala ou escuta



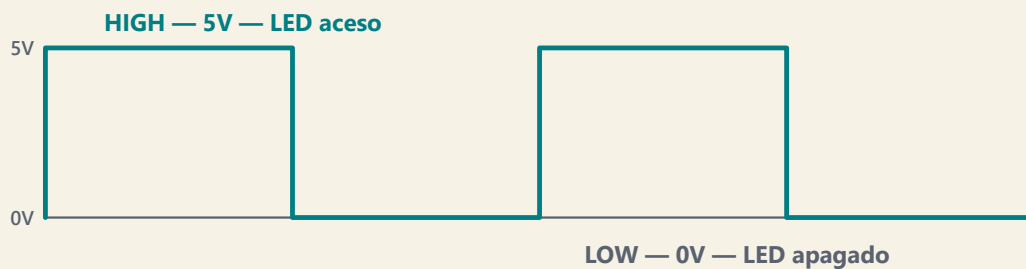
```
pinMode(pino, modo); // modo: OUTPUT, INPUT ou INPUT_PULLUP
```

Onde vai	Sempre dentro do <code>setup()</code>
OUTPUT	o pino passa a enviar 0V ou 5V — para LED, buzzer
INPUT	o pino passa a ler — para botão, sensor digital
INPUT_PULLUP	igual a <code>INPUT</code> , mas liga um resistor interno (~20–50 kΩ) até o 5V

⚠ **Esqueceu o `pinMode`?** O pino fica como entrada por padrão e o LED simplesmente não acende — sem nenhuma mensagem de erro. É o bug silencioso mais comum do curso.

Pinos **analógicos usados como entrada analógica não precisam** de `pinMode`.

B.3 • `digitalWrite()` — ligar e desligar



```
digitalWrite(pino, HIGH); // manda 5V
digitalWrite(pino, LOW);  // manda 0V
```

```
digitalWrite(pino, HIGH); // manda 5V -> LED acende
digitalWrite(pino, LOW);  // manda 0V -> LED apaga
```

⚠ `HIGH` e `LOW` são **maiúsculas**. `high` não compila. E o pino precisa ter sido declarado como `OUTPUT` no `setup()`.

B.4 • `digitalRead()` — ler liga/desliga

```
int estado = digitalRead(pino); // devolve HIGH (1) ou LOW (0)
```

0**LOW · 0V**

Botão solto / claro

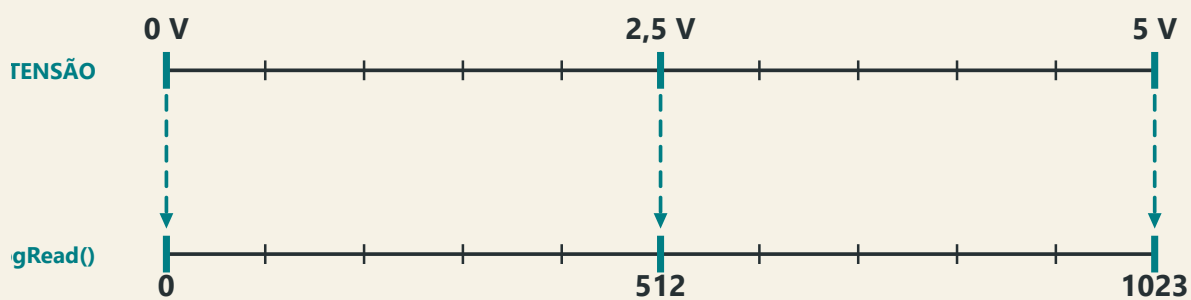
1**HIGH · 5V**

Botão apertado / escuro

`digitalRead()` devolve só isso: 0 ou 1. Nada no meio.

Use para botão, chave fim de curso, sensor de presença — qualquer coisa que só tenha **dois estados**. Não serve para o LDR: luz não é liga/desliga, é uma escala, e para escala usamos `analogRead()`.

B.5 · `analogRead()` — o robô ganha um sentido



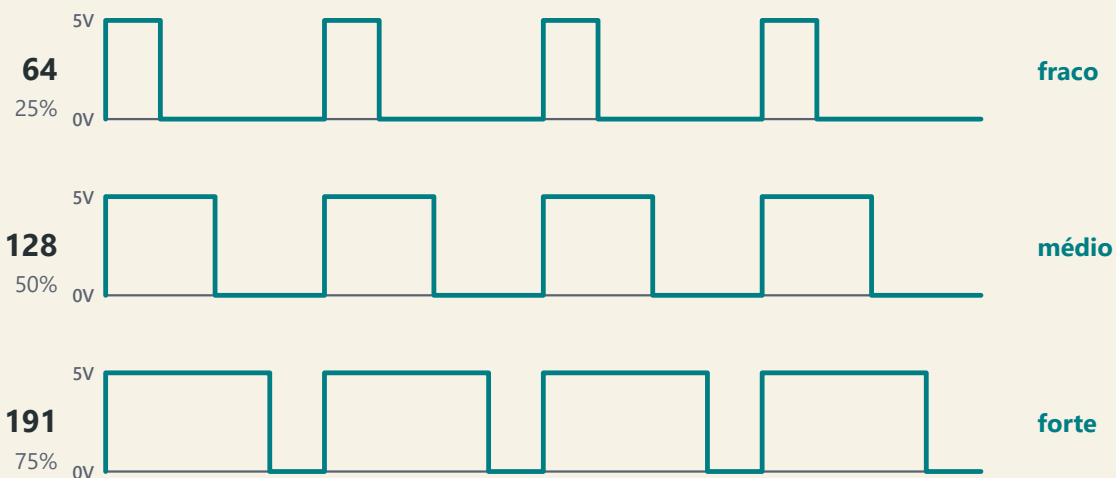
conversor de 10 bits: $2^{10} = 1024$ níveis · cada nível $\approx 4,9$ mV

```
int leitura = analogRead(A0); // devolve 0 a 1023
```

Lê	Devolve
0 V	0
2,5 V	512
5 V	1023

⚠ Só funciona nos pinos **A0 a A5**. E o valor **não** é a resistência do sensor — é a tensão no divisor. Por isso cada montagem precisa ser **calibrada**.

B.6 • `analogWrite()` — PWM, o falso meio-termo



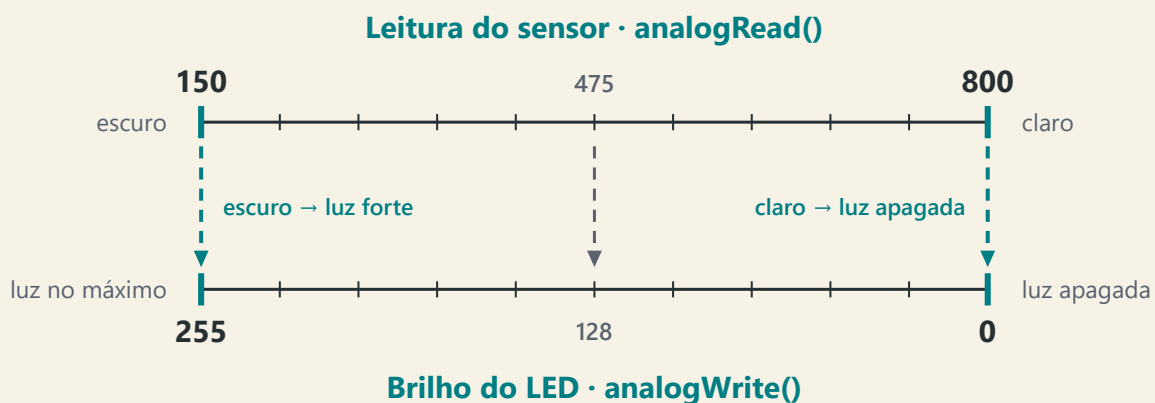
O pino continua só ligando e desligando — mas tão rápido (~490 Hz) que o olho enxerga a MÉDIA.

```
analogWrite(pino, valor); // valor: 0 (apagado) a 255 (máximo)
```

⚠ **Só funciona nos pinos com ~** : 3, 5, 6, 9, 10, 11. Num pino comum, `analogWrite` vira um liga/desliga sem meio-termo — é o bug nº 1 do projeto final.

Repare na assimetria: `analogRead` vai até **1023**, `analogWrite` até **255**. É exatamente por isso que existe o `map()`.

B.7 • `map()` e `constrain()` — traduzir faixas



```
map(valor, deMin, deMax, paraMin, paraMax); // converte de uma faixa para outra
constrain(valor, minimo, maximo);           // trava o valor dentro dos limites
```

No projeto final:

```
int brilho = map(leitura, ESCURO, CLARO, 255, 0);
brilho = constrain(brilho, 0, 255);
```

Leitura **150** (escuro) → brilho **255** (luz no máximo). Leitura **800** (claro) → brilho **0** (luz apagada).

⚠ **map()** **não trava nos limites**. Se a leitura sair da faixa calibrada, o resultado passa de 255 ou fica negativo — e o LED pisca errado. Por isso o **constrain()** vem **sempre logo depois**.

map() usa matemática **inteira**: `map(5, 0, 10, 0, 3)` dá **1**, não **1,5**.

B.8 • `delay()` e `millis()` — o tempo

`delay(1000)`

o programa PARA por 1 segundo

nada acontece

Durante o `delay` o Arduino não lê sensor, não responde a botão, não faz mais nada.

`millis()`

devolve há quanto tempo ligou



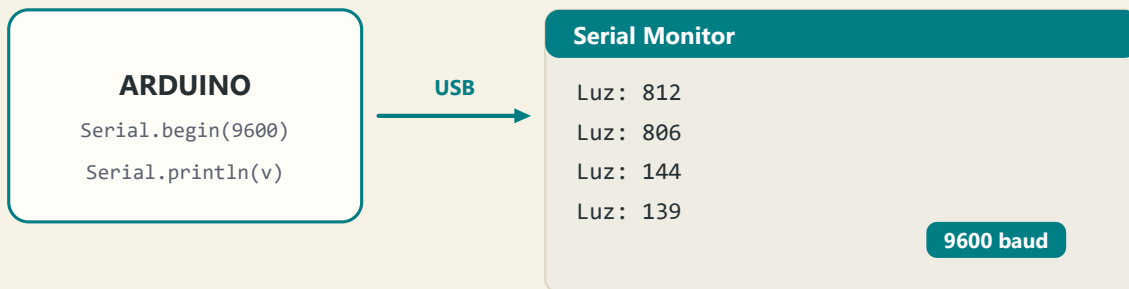
O programa continua rodando: você só compara os tempos e decide quando agir.

`delay()` é mais simples; `millis()` é o que permite fazer duas coisas ao mesmo tempo.

```
delay(1000);           // pausa 1000 ms — BLOQUEIA tudo
unsigned long t = millis(); // ms desde que a placa ligou — não bloqueia nada
```

Por que isso importa para robótica: com `delay()`, o robô só faz uma coisa por vez. Com `millis()`, ele pisca um LED, toca um alarme, lê três sensores e move um motor "ao mesmo tempo". É o passo que separa o exercício do projeto real.

B.9 • Serial — a janela para dentro do Arduino



O número embaixo do monitor tem que ser o mesmo do Serial.begin()

```
Serial.begin(9600);           // no setup(): abre o canal com o PC
Serial.print("Luz: ");       // imprime, sem pular linha
Serial.println(leitura);     // imprime e PULA linha
```

Situação	O que fazer
Aparecem símbolos malucos (??x??)	o baud do monitor não bate com o <code>Serial.begin()</code>
Não aparece nada	faltou <code>Serial.begin(9600);</code> no <code>setup()</code>
Rola rápido demais	aumente o <code>delay()</code> no fim do <code>loop()</code>
Upload falha com o monitor aberto	feche o Serial Monitor e envie de novo

Serial é o seu multímetro. Sempre que não souber o que o robô está "pensando", mande ele imprimir. Depurar às cegas custa 10× mais tempo.

B.10 • tone() e noTone() — som

```
tone(pino, frequencia);      // toca até mandar parar
tone(pino, frequencia, duracao); // toca por N ms e para sozinho
noTone(pino);                // silencia
```

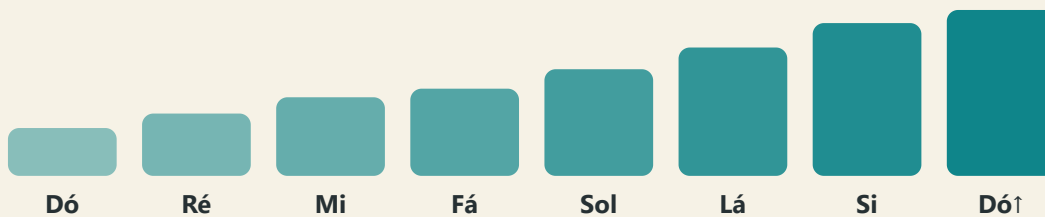
Nota	Dó	Ré	Mi	Fá	Sol	Lá	Si	Dó↑
Hz	262	294	330	349	392	440	494	523

⚠ **tone() com duração NÃO pausa o programa.** A linha seguinte executa na hora. Sem um `delay()` do mesmo tamanho logo depois, a melodia sai toda atropelada. Este é o bug clássico da Prática 3.

`tone()` usa um temporizador interno que também controla os pinos **3 e 11** — enquanto o som toca, o PWM desses dois pinos para de funcionar.

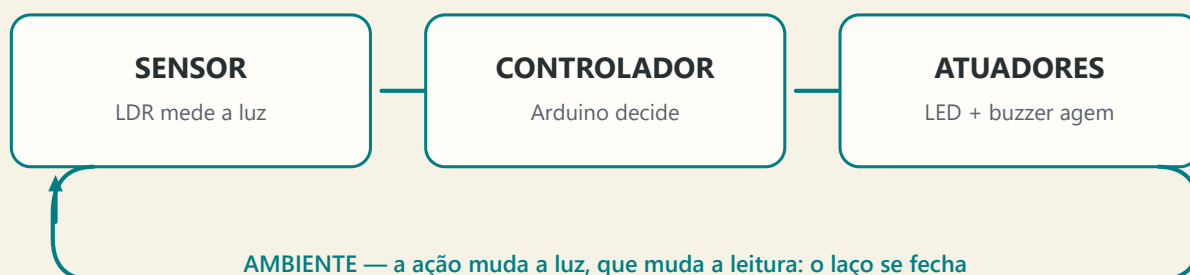
frequência (Hz)

mais agudo →



```
tone(pino, 440); // toca lá até mandar parar
tone(pino, 440, 300); // toca 300 ms e para sozinho
```

B.11 · Colando tudo: o fluxo do projeto final



B.12 · Cartão de erros — leia antes de chamar o instrutor

Sintoma	Causa mais provável
LED não acende	perna invertida · faltou <code>pinMode</code> · faltou o GND
LED aceso fraco e sempre igual	<code>analogWrite</code> num pino sem <code>~</code>
Brilho invertido (acende no claro)	ordem trocada no <code>map()</code>
Sensor travado em 0 ou 1023	divisor mal montado — conferir 5V, A0 e GND
Valores "pulando" sozinhos	fio solto, ou o pino A0 está no ar
Melodia atropelada	faltou <code>delay()</code> depois do <code>tone()</code>
Alarme dispara sempre	limiar não calibrado naquela mesa
Símbolos malucos no monitor	baud diferente do <code>Serial.begin()</code>
<code>expected ';'</code>	ponto e vírgula faltando — olhe a linha anterior
<code>'x' was not declared</code>	erro de digitação ou maiúscula trocada
Nenhuma porta em Tools → Port	cabo USB só de carga , sem dados